Django Backend Guide for a React Music App

How to learn Django and apply it to music functions like upload, playback, next/previous, mix, and accounts.

Overview

This guide is written for a React music player app that needs a Django backend for user accounts, song upload/storage, library retrieval, playback support, and playlist behavior.

In Django, you will build models to store metadata, REST endpoints to manage and serve data, and authentication/permissions to protect user content.

Core Django Concepts to Learn

Django project and app structure: how to create a project, add apps, and connect the database.

Models and migrations: define tables using models.py and apply schema changes with migrations.

Django REST Framework (DRF): build serializers, views, viewsets, and routers for JSON APIs.

Authentication and permission classes: protect endpoints and restrict data by user account.

Media file upload and storage: configure MEDIA_ROOT, MEDIA_URL, and FileField/FileSystemStorage or cloud storage.

User Accounts

Use Django’s built-in auth system for User models, plus optionally extend with a Profile model for app-specific fields.

For API auth, learn DRF TokenAuth, SessionAuth, or JWT (Simple JWT) so the React app can log in and call protected endpoints.

Account functions include signup, login, logout, password reset, user profile, and user-specific libraries.

Models for the Music App

Song model: title, artist, album, owner (user), audio file field, cover image, duration, audio format, created timestamp.

Playlist model: name, owner, many-to-many relationship to songs, order field or through model for sequence.

Playback queue model (optional): current song, queue items, order index, and status fields for active sessions.

UserProfile model: optional fields like subscription tier, preferences, or upload limits.

Uploading Songs

Use a POST endpoint such as /api/songs/ that accepts multipart/form-data with the audio file and metadata.

In Django model, use FileField or models.FileField(upload_to="songs/%Y/%m/%d/").

Configure MEDIA_ROOT and MEDIA_URL in settings.py so Django stores files under a local folder and serves media during development.

For production, switch to a cloud storage backend (Amazon S3, Google Cloud Storage, Azure) or a dedicated static/media server.

The upload flow in React: user selects file, sends FormData to the API, backend validates user ownership and saves the file.

Where Uploaded Files Are Kept

During development: store files locally in MEDIA_ROOT, for example /media/songs/... .

In production: store files in object storage and use signed URLs for secure playback if needed.

Store file path or URL in the Song model. Keep metadata in the database and file content on disk/cloud.

Protect direct file access so one user cannot retrieve another user’s files unless allowed.

Retrieving Songs and Library Data

Create read endpoints such as /api/songs/, /api/songs//, and /api/users//songs/.

Use serializer classes to convert Song objects into JSON with title, artist, play count, audio URL, and cover image URL.

Filter songs based on request.user so each account sees only their own uploads or permitted content.

React consumes these endpoints to display the library, playlists, and song cards.

Playing Songs

Playback is usually managed by the React client, but Django must provide a valid audio URL or stream endpoint.

Use MEDIA_URL for direct access to uploaded files in development, or a DRF view that returns a signed URL or protected stream in production.

Example endpoint: /api/songs//stream/ could return the file path or stream chunked audio data.

If using direct file URLs, ensure permissions prevent unauthorized access and that URLs are only returned to logged-in users.

Next / Previous Functionality

Store song order in a playlist or queue, and use current track index to compute next and previous IDs.

The React app can keep local state for the active playlist and call /api/songs// or /api/playlist//items/ to load the next track.

Backend support can include endpoints like /api/queue/next/ and /api/queue/prev/ if you want the server to manage current playback state.

Design the API so the client can request the next/previous song metadata and audio URL without reloading the entire library.

Mix and Shuffle

Mix can mean a randomized queue, a shuffle playlist, or special recommendations. For shuffle, randomly order the song list or playlist items.

A backend mix endpoint could return a shuffled list of songs: /api/songs/shuffle/ or /api/playlists//shuffle/.

If the app supports crossfade, keep that in client logic and use the backend only to supply the ordered song list.

For advanced mixing, track play history and generate a “smart mix” based on genre, artist, or most played songs.

Account-Specific Behavior

Every song upload should include owner=request.user. That ensures each account only sees its own library unless you build sharing features.

Use Django permissions so only the owner or collaborators can edit, delete, or retrieve a song record.

Account endpoints can include /api/me/, /api/users//profile/, and /api/users//library/.

For shared playlists, use separate models and permission rules to allow multiple users to access the same playlist.

How This Applies to Your React App

Upload button: send FormData with audio file and metadata to Django POST /api/songs/.

Library page: fetch /api/songs/ and render cards. Use user auth token to show only your uploads.

Player controls: request song metadata and audio URL from Django, then play in the React audio element.

Next/prev: update React queue state, and optionally call server endpoints to confirm ordering or retrieve next track info.

Shuffle: either request a shuffled list from Django or shuffle client-side based on the current library.

Recommended Learning Path

Start with Django basics: models, views, URLs, templates, and settings.

Learn Django REST Framework for API design, serializers, viewsets, and routers.

Study authentication flows: login, signup, token/JWT, and permission classes.

Practice file upload and storage in Django, then migrate to cloud storage if needed.

Build the app incrementally: accounts first, then upload, then library retrieval, then playback and shuffle.

Example Model Snippets

```
class Song(models.Model):

owner = models.ForeignKey(settings.AUTH_USER_MODEL,
on_delete=models.CASCADE)

title = models.CharField(max_length=200)

artist = models.CharField(max_length=200)

audio = models.FileField(upload_to="songs/%Y/%m/%d/")

cover = models.ImageField(upload_to="covers/%Y/%m/%d/", blank=True,
null=True)

created_at = models.DateTimeField(auto_now_add=True)
```

Practical Project Steps

1. Set up Django and create a project and app.

2. Create UserProfile and Song models, then run migrations.

3. Install DRF and build serializers and viewsets for song upload and retrieval.

4. Configure media settings and test local file uploads with a React form.

5. Add authentication endpoints and connect React auth flows.

6. Build the React player to fetch song metadata and play audio from the backend.

7. Add playlist, next/prev, and shuffle endpoints as needed.

8. Deploy the backend and storage to production once the flow works locally.